NNES Reference and Known-Truth Validation Tutorial

Neural Network Efficient Estimator

econirl package documentation

May 2026

RTD page: https://econirl.readthedocs.io/en/latest/estimators/nnes.html **Source:** https://github.com/rawatpranjal/EconIRL/tree/main

1 Purpose

NNES estimates dynamic discrete choice models with a neural approximation to the integrated value function. The motivation is the same structural target as NFXP, CCP, MPEC, and SEES: recover reward parameters, the implied policy, the value function, and counterfactual policies. The difference is that NNES uses a neural value function inside an NPL-style policy iteration step, so it can in principle scale beyond low-dimensional tabular state spaces.

This document is a known-truth validation tutorial. It does not use real data. The synthetic DGP supplies the true reward, transition law, value function, policy, and Type A/B/C counterfactual oracle objects. The package implementation runs on two DGP cells and must pass the hard structural recovery gates before the generated results are written. The low-dimensional cell is a sanity check. The high-dimensional cell is the main NNES validation because it is the setting where the neural value approximation is meant to add value over fully tabular estimators.

2 Source Paper

The source paper is Nguyen (2025). The paper embeds a neural value function approximation in the Nested Pseudo-Likelihood framework of Aguirregabiria and Mira (2002). The key theoretical result is a zero-Jacobian or Neyman-orthogonality property: at the true conditional choice probabilities, first-order value-network approximation error drops out of the structural score.

Under the paper's regularity conditions, if the neural value approximation converges faster than $n^{-1/4}$, the structural parameter estimator is $\sqrt{n}$-consistent and efficient. The orthogonality result is powerful, but it is not a finite-sample guarantee. The implementation still has to put the neural value function, policy update, reward parameters, and Bellman level on a mutually consistent point of the model.

3 When to Use NNES

Use NNES when the model is structural, transitions are known or first-stage estimated, the reward is parametric, and the state space is large enough that exact tabular value iteration or full MPEC constraints are impractical. NNES is not a behavioral cloning shortcut: matching observed choices is insufficient unless the recovered reward and counterfactual policies also match the structural model.

Use the generated results below as the package-level recovery check for the finite-state known-truth benchmark. The shared harness checks more than the training loss or in-sample choice fit: it verifies parameter, reward, policy, value, Q, and counterfactual recovery.

4 Model

Table 1: Common DDC notation shared across all primers.

Symbol	Name	Definition
$s \in \mathcal{S}$	State	Discrete or continuous
$a \in \mathcal{A}$	Action	Finite: $\lvert\mathcal{A}\rvert = J$
s'	Next state	Drawn from $P(s' \mid s, a)$
$\beta \in [0, 1)$	Discount factor	
$\sigma > 0$	Scale parameter	Logit noise scale (= 1 by convention)
$\theta \in \Theta \subset \mathbb{R}^{d_\theta}$	Structural parameters	Unknown
$\phi(s, a)$	Feature vector	Known functions of (s, a), dimension K
$u(s, a; \theta)$	Flow utility	$\theta^\top \phi(s, a)$ (linear-in-parameters)
$P(s' \mid s, a)$	Transition kernel	Known or estimated from data
$Q(s, a)$	Choice-specific value	$u(s, a; \theta) + \beta \sum_{s'} P(s'\lvert s, a)\, V(s')$
$V(s)$	Integrated value	$\sigma \log \sum_a \exp(Q(s, a)/\sigma)$
$\pi(a \mid s)$	Choice probability	$\exp\big((Q(s, a) - V(s))/\sigma\big)$
τ	Trajectory	$(s_0, a_0, s_1, a_1, \ldots)$ from one agent
$\mathcal{D}$	Panel data	Collection of N observed trajectories

The observed data are state-action-next-state trajectories

$$\{(s_{it}, a_{it}, s_{i,t+1}) : i = 1, \ldots, N,\ t = 1, \ldots, T\}.$$

Flow payoff is linear in known features,

$$u_\theta(s, a) = \phi(s, a)^\top \theta,$$

with transition law $P_a(s, s')$, discount factor β, and type-I extreme-value scale σ. Given a continuation value V,

$$Q_\theta(s, a; V) = u_\theta(s, a) + \beta \sum_{s'} P_a(s, s') V(s'),$$

and the implied logit policy is

$$\pi_{\theta,V}(a \mid s) = \frac{\exp(Q_\theta(s, a; V)/\sigma)}{\sum_b \exp(Q_\theta(s, b; V)/\sigma)}.$$

5 NNES Estimator

Given conditional choice probabilities $P(a \mid s)$, the finite-state NPL evaluation step profiles the value representation as

$$W_\theta[P] = W_z[P]\theta + W_e[P],$$

where $W_z[P]$ is the expected discounted feature valuation and $W_e[P]$ is the expected discounted logit entropy term under the fixed CCP iterate. The implementation trains a ReLU MLP $V_\gamma(s)$ by supervised regression on $W_\theta[P]$ as the NNES value approximation, then optimizes the structural parameters through the profiled one-step NPL likelihood rather than freezing the continuation value at the previous θ. The reported validation value function is re-evaluated from the final recovered reward and policy on the Bellman scale.

Algorithm 1: NNES with NPL value regression

Input: Panel data, reward features, transitions, discount β, initial CCPs, anchor state
Estimate initial CCPs from empirical state-action frequencies
for *outer iteration* $k = 1, \ldots, K$ **do**
  Compute the NPL value target $W_\theta[P_{k-1}]$
  Normalize the target by the anchor state
  Train V_γ to predict the anchored target over all states
  Maximize the profiled pseudo-likelihood over θ through $W_z[P]\theta + W_e[P]$
  Update CCPs from the implied logit policy
end
Evaluate the final policy value on the Bellman scale
Output: $\hat{\theta}$, value function, policy, log likelihood, diagnostics

Comparison with SEES. SEES (Luo and Sang, 2024) approximates V with deterministic basis functions and penalizes the Bellman residual in a joint optimization problem. NNES uses a neural value approximation and NPL orthogonality instead. In principle, this avoids the Schur-complement information matrix needed by SEES. In the finite-state validation harness, the profiled NPL step gives an exact policy-evaluation specialization of the paper's neural approximation idea.

6 Known-Truth DGPs

The generator runs two cells. The first is `canonical_low_action`, the same low-dimensional action-dependent structural benchmark used as a sanity check for NFXP, CCP, MPEC, and SEES. The second is `canonical_high_action`, a higher-dimensional action-dependent benchmark with richer state and reward features. This second cell is the more important NNES result: it tests whether the estimator still recovers structural objects when the state representation and reward basis are no longer tiny.

Table 2: Canonical NNES validation DGPs.

Quantity	Low-dimensional	High-dimensional
Regular states	20	80
Absorbing states	1	1
Total states	21	81
State-feature dimension	2	16
Reward parameters	4	32
Actions	3	3
Exit action	2	2
Discount factor	0.95	0.95
Shock scale	1.0	1.0
Reward mode	action-dependent	action-dependent
Simulated individuals	2,000	2,000
Periods per individual	80	80
Observations	160,000	160,000

The validation gates check parameter recovery, reward recovery, baseline policy recovery, Bellman-level value and Q recovery, and Type A/B/C counterfactual regret. The NNES generator runs with enforced gates, so this section is generated only when all checks pass.

7 Generated Validation Results

This section is generated by `nnes_run.py` from the `experiments.known_truth` harness. Hard gates are enforced. The low-dimensional cell is a sanity check; the high-dimensional cell is the main NNES validation because it

exercises the larger state and reward-feature setting where the neural estimator is meant to matter.

7.1 Validation Design

Both cells use 2,000 simulated individuals, 80 periods per individual, known transitions, three actions, homogeneous rewards, and the shared Type A/B/C counterfactual oracles. The high-dimensional cell is the primary stress test; the low-dimensional cell checks that the same implementation still behaves correctly in the easy benchmark.

Table 3: NNES known-truth validation cells.

Cell	Role	States	State dim.	Reward params	Obs.	Gates
Low-dimensional	sanity check	21	2	4	160,000	11/11
High-dimensional	primary validation	81	16	32	160,000	11/11

The high-dimensional DGP passes the pre-estimation checks before NNES is fit: feature rank is 32/32, the reward-feature condition number is 1.377, all 81 states are observed, and state-action coverage is 0.959.

7.2 Fit Summary

Table 4: NNES fit summary.

Cell	Converged	L-BFGS-B iter.	Final V loss	Log likelihood
`canonical_low_action`	true	10	0.0004	-174875.8
`canonical_high_action`	true	29	0.0299	-160272.9

The easy cell is very tight. The high-dimensional cell is deliberately harder: reward, value, and Q errors are larger, but they remain within the hard gates.

7.3 Main Recovery Numbers

Table 5: Core recovery metrics.

Metric	Gate	Low-dimensional	High-dimensional
Parameter cosine similarity	≥ 0.95	0.9982	0.9912
Parameter relative RMSE	≤ 0.30	0.0652	0.1351
Reward RMSE	≤ 0.08	0.0102	0.0640
Policy total variation	≤ 0.03	0.0056	0.0238
Value RMSE	≤ 0.20	0.0198	0.1156
Q RMSE	≤ 0.20	0.0234	0.1371

7.4 Counterfactual Recovery

Each counterfactual gate requires regret below 0.05. All three counterfactuals pass in both cells.

Table 6: Counterfactual regret.

Counterfactual	Low-dimensional	High-dimensional	Interpretation
Type A	0.0002	0.0049	reward/state shift
Type B	0.0003	0.0056	transition change
Type C	0.0001	0.0013	action restriction

Exact parameter vectors, standard errors, log-likelihoods, policy distances, and all gate values are written to `nnes_results.json`. The PDF reports the decision-critical values rather than printing the full 32-parameter high-dimensional vector.

7.5 Gate Audit

Table 7: Hard-gate audit by cell.

Cell	Passed gates	Failed gates
`canonical_low_action`	11	0
`canonical_high_action`	11	0

8 Interpretation

The validation runs confirm that the NNES implementation recovers the same structural objects checked for the other validated estimators: reward parameters, policy, Bellman-level values, Q values, and counterfactual policies. The low-dimensional result shows that the implementation works on the easy case. The high-dimensional result is the substantive NNES evidence: it shows that the profiled NPL/neural-value path remains inside all hard gates with 81 states, 16-dimensional state features, and 32 reward parameters.

The key implementation detail is that the structural MLE step profiles policy evaluation through θ, matching the source paper's one-step NPL construction. This avoids freezing the continuation value at the previous parameter vector during the likelihood step.

9 Reproduce

From the repository root:

```
PYTHONPATH=src:. python papers/econirl_package/primers/nnes/nnes_run.py --quiet-progress
```

By default the script runs both the low-dimensional sanity check and the high-dimensional primary validation. It writes `nnes_results.tex`, `nnes_results.json`, and full JSON copies under:

```
/tmp/econirl_nnes_primer_known_truth
```

References

Victor Aguirregabiria and Pedro Mira. Swapping the nested fixed point algorithm: A class of estimators for discrete Markov decision models. *Econometrica*, 70(4):1519–1543, 2002.

Yao Luo and Peijun Sang. Efficient estimation of structural models via sieves. Working Paper, University of Toronto, 2024.

Hoang Nguyen. Neural networks for efficient estimation of high-dimensional dynamic discrete choice models. Working Paper, Georgetown University, 2025.